# Python Programming

## 9 Essential Topics for University Students

Data types · OOP · Operators · Decisions · Loops

Name: CHENG Sokvisal
ID: e20241146
Group: I3-AMS-1A

Course: Object-Oriented Programming (OOP)
Lecturer: Mr. VANN Visal

# Overview

- Nine topics
- Every concept: definition, code, output
- Python — all examples are runnable Python 3.10+

# Learning Content

- Python Basics Data Types
- Python Variables and Attributes
- Python Modifiers and Access Control — I & II
- Python Operators — I & II
- Python Decision-Making Statements — I & II
- Python Loops and Loop Control

TOPIC 1

# Python Basic Data Types

Primitive/Built-in Data Types, Collections and Reference Types and Literals in Python

# 1: Variables Hold Different Types

```
student_name = "Sok Visal"
age = 20
gpa = 3.25
is_enrolled = True
subjects = ["Math", "Statistics", "Python"]
print(type(student_name)) //String
print(type(age)) //integer
print(type(gpa)) //float
print(type(is_enrolled)) //boolean
print(type(subjects)) //list
```

- Primitive/Built-in data types refers to basic types of data that Python can work with. They tell Python what kind of value a variable stores.
- A data type tells Python what a value is and what you can do with it.
- type() reports the type.
- Python infers the type from the value.

**Output**

```
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>
<class 'list'>
```

# 1: tuple, dict, set; Mutable vs Immutable

```
point = (3, 4)
info = {"name": "Visal", "age": 20} //dictionary
tags = {"a", "b", "a"} //Set
subjects = ["Math"] //List
subjects.append("Python") //Add “Python” to
subjects
print(subjects, len(tags))
point[0] = 9 //it can not change the number in
point because it’s tuple, since tuple is a fixed
result that cannot be changed
```

- “Collection Types” are data types that can store or organize multiple values.
- “Reference Types” relates to how variables refer to objects in memory.
- list, dict, set are mutable (changeable)
- tuple, str, int are immutable
- set drops duplicates

**Output**

```
['Math', 'Python'] 2
TypeError: 'tuple' object does not
support item assignment
```

# 1: Type Conversion and Common Errors

```
print(int("25") + 1) //26
print(float(3)) //3.0
print(str(99) + "%") //99%
print(bool(0), bool("hi")) //False, True
print("Age: " + 20) //Error, Since Age: is
string and 20 is integer so it cannot be Added
together
```

- literal is the actual value itself, not a variable name. Ex: numeric, string, Boolean, None, collection literals
- int(), float(), str(), bool() convert values
- input() returns str: convert before math
- Mistake: mixing str and int

**Output**

```
26
3.0
99%
False True
TypeError: can only concatenate
str (not "int") to str
```

# Practical Example: Storing Student informations

```
students = [
        {"name": "Visal", "age": 19, "score": 85},
        {"name": "Dara", "age": 20, "score": 92},
]

for student in students:
        print("Name:", student["name"])
        print("Age:", student["age"])
        print("Score:", student["score"])
```

- First, we create a dictionary called “students” that contains name, age and score to determine the student’s name, age and score.
- For the loop, we takes each dictionary from the students list, one at a time, and assigns it to the variable student. This continues until Python has processed every record.

**Output**

```
Name: Visal
Age: 19
Score: 85

Name: Dara
Age: 20
Score: 92
```

# 1: Data Types at a Glance

| Type | Example | Mutable? | Ordered? |
|---|---|---|---|
| int | 20 | No | – |
| float | 3.25 | No | – |
| bool | True | No | – |
| str | "Visal" | No | Yes |
| list | [1, 2] | Yes | Yes |
| tuple | (1, 2) | No | Yes |
| dict | {"a": 1} | Yes | Insertion order |
| set | {1, 2} | Yes | No |

# 1: Key Takeaways

- Every value has a type; check with type()
- Convert explicitly with int(), float(), str(), bool()
- Lists, dicts, sets change; tuples and strings do not
- Common error: combining str and int

TOPIC 2

# Variables and Attributes

Local · global · instance · class attributes

# 2: Local vs Global Variables

```
count = 0 //Global variable
def add_one(): //Define a function
    global count //count refers to global
    count += 1 //increase count by one
    temp = 5 //local variable
add_one() //calls function
print(count)
print(temp) //NameError because temp is local
variable inside add_one()
```

- Local variable is a variable belongs to the function where it is created
- Global variable is a variable that can be used in multiple places in the program, rather than being limited to one function
- temp is local: exists only inside the function
- global lets a function reassign a global name
- Names: snake_case, no spaces, not a keyword

**Output**

```
1
NameError: name 'temp' is
not defined
```

# 2: Changing Attributes

```
class Student:   //class Student is like a blueprint
    school = "Institute“  //school is class attribute
    def __init__(self, name, age):
        self.name = name
        self.age = age
s1 = Student("Visal", 20) //object 1
s2 = Student("Dara", 21) //object 2
s1.age = 21 //change object1’s age to 21
Student.school = "Tech Institute“ //changing class attribute
s2.school = "Private" //s2’s school now is “Private”
print(s1.age, s1.school)
print(s2.school, Student.school)
```

- Instance attribute stores information about an individual object.
- Instance change affects one object
- Class change affects all objects without their own copy
- s2.school creates a new instance attribute that shadows the class one
- __init__=initializes, in this code we use it to initializes each new student object. Ex: self.name is instance attribute named “name” same as age.

**Output**

```
21 Tech Institute
Private Tech Institute
```

# Practical Example: The Student Class

```
class Student:
    school = "Institute of Technology"

    def __init__(self, name, age):
        self.name = name
        self.age = age

student1 = Student("Visal", 20)
student2 = Student("Dara", 21)

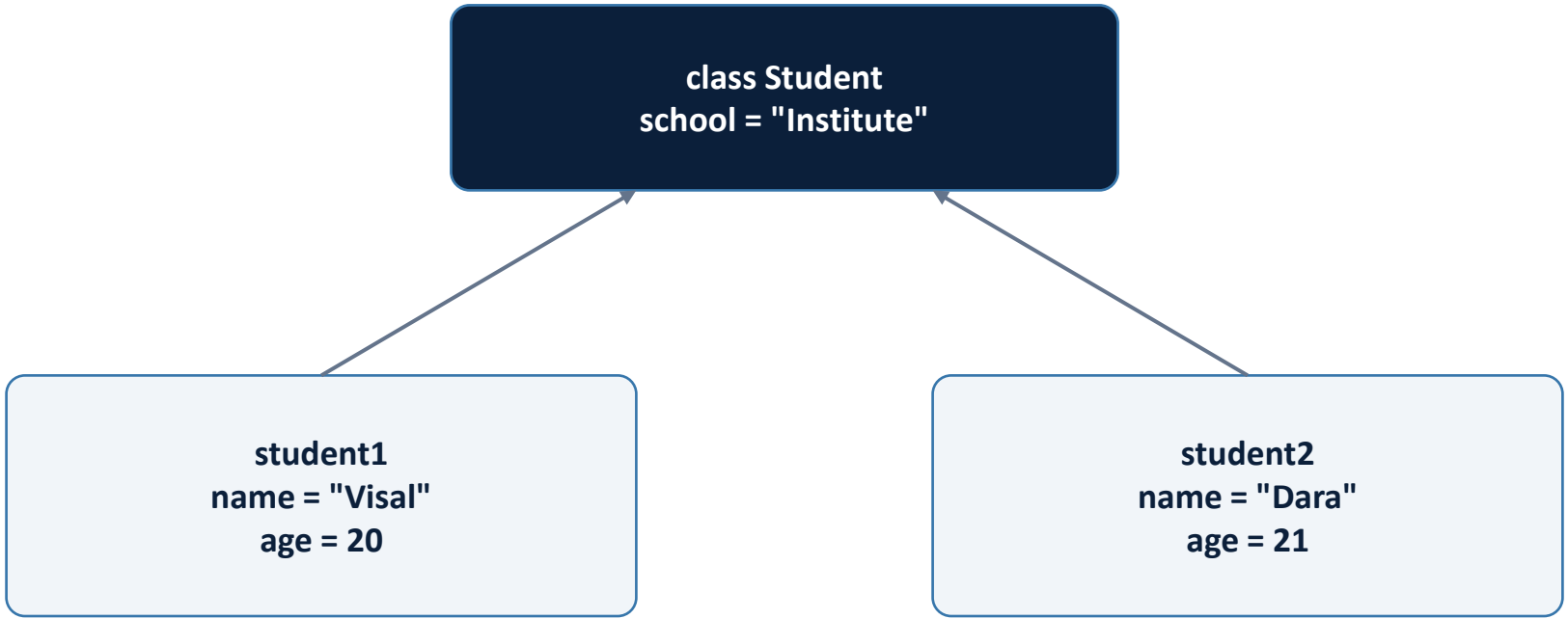
print(student1.name)
print(student2.name)
print(student1.school)
```

- school: class attribute, shared
- __init__ runs when an object is created
- self = the object itself; self.name is its own attribute

**Output**

```
Visal
Dara
Institute of Technology
```

# 2: Object Diagram




Instance attributes: one set per object. Class attribute: one shared value.

# 2: Key Takeaways

- Variables: names in functions or modules; attributes: names on objects
- Local vs global scope; use global sparingly
- Instance attributes belong to one object; class attributes are shared
- Assigning through an instance shadows the class attribute

TOPIC 3

# Modifiers and Access Control — Part 1

Naming conventions · name mangling · @staticmethod · @classmethod

# Practical Example: BankAccount Class

```
class BankAccount: //define class called BankAccount
    bank_name = "ABC Bank“ //class attribute called bank_name

    def __init__(self, owner, balance): //constructor
        self.owner = owner
        self._balance = balance
        self.__account_id = "ACC001"

    def display_balance(self): //instance method to receive self
        return self._balance

    @staticmethod //receives only the arguments you provide
    def calculate_interest(balance, rate):
        return balance * rate

    @classmethod //receives the class itself as its first argument
    def get_bank_name(cls):
        return cls.bank_name
```

- @staticmethod performs a task related to the class but doesn’t automatically receive the object or class
- @classmethod works with the class itself
- owner: public
- _balance: non-public by convention
- __account_id: name-mangled
- Static: no self/cls. Class: receives cls

**Output**

```
(class definition)
```

# 3: Using It

```
acct = BankAccount("Visal", 1000) //Creates object
print(acct.owner) //owner is public attribute
print(acct.display_balance()) //public method
print(acct.calculate_interest(1000, 0.05))
print(BankAccount.get_bank_name()) //@classmethod
print(acct._BankAccount__account_id)
print(acct.__account_id) //AttributeError because
python transforms __account_id into
_BankAccount_account_id inside the class
```

- Public, Non-Public, and Private Naming Conventions describe how we name attributes and methods to indicate whether they are intended to be accessed from outside a class. Ex: Public (owner), Non-public (_id), Private (__account)
- Mangling turns __x into _Class__x
- Python does not enforce privacy; avoid touching mangled names
- Last line fails: AttributeError

**Output**

```
Visal
1000
50.0
ABC Bank
ACC001
AttributeError
```

# 3: Three Kinds of Methods

| Kind | First parameter | Reaches | Use for |
|---|---|---|---|
| Instance method | self | Object and class data | Behaviour of one object |
| @classmethod | cls | Class data | Shared settings, alternative constructors |
| @staticmethod | (none) | Nothing automatically | Helper that belongs with the class |

# 3: Key Takeaways

- name = public; _name = internal by convention; __name = mangled
- Python trusts programmers: conventions, not enforcement
- Use @classmethod for class data, @staticmethod for helpers
- Use methods to control changes to internal data

TOPIC 4

# Python Modifiers and OOP Features

Abstract Classes and Methods · Access Control and Encapsulation · typing.final · @property · @final

# 4: Abstract Class

```
from abc import ABC, abstractmethod //ABC allows us to define
an abstract base class

class Shape(ABC): //This creates an abstract class called Shape
    @abstractmethod //marks area() as an abstract method
    def area(self): //refers to the current object
        pass //means the method has no implementation here

class Circle(Shape): //defines Circle as a child class of Shape
    def __init__(self, radius): //constructor
        self.radius = radius

    def area(self): //Implement the abstract method
        return 3.14159 * self.radius ** 2

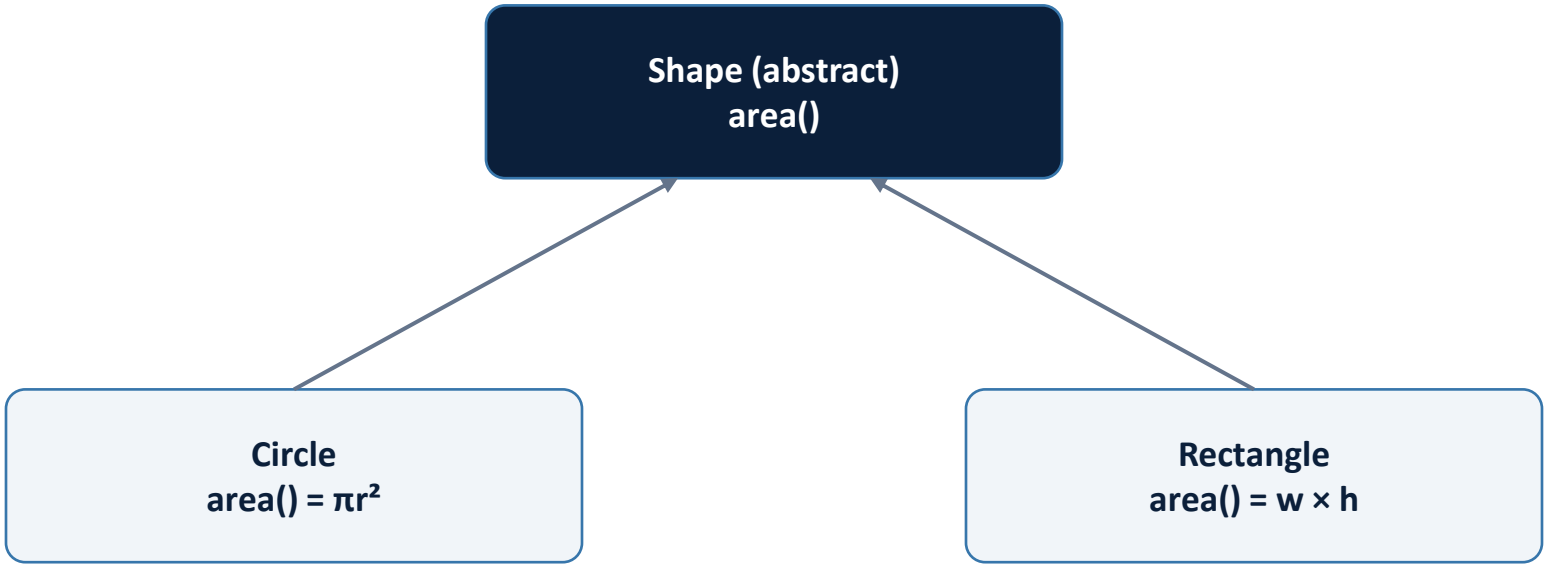
print(Circle(2).area())
shape = Shape() //TypeError because Shape has an abstract
method, area() that has not been implemented
```

- Abstract class is a class that acts as a blueprint for other classes.
- Abstract method is a method declared in an abstract class that must be implemented by a concrete child class before you can create an object of that child class.
- Shape promises area() but does not implement it
- Circle implements it, so it can be instantiated
- Shape() fails: abstract methods remain

**Output**

```
12.56636
TypeError: Can't instantiate
abstract class Shape ...
```

# 4: Abstract Parent, Concrete Children




Subclasses must implement every abstract method.

# 4: Encapsulation with @property

```
class Student: //create the Student class
    def __init__(self, score): //runs automatically when create
a new Student object
        self.score = score

    @property //allows to access the method as if it were an
attribute
    def score(self):
        return self._score

    @score.setter //controls what happens when someone assigns
a new value to score
    def score(self, value):
        if not 0 <= value <= 100:
            raise ValueError("Score must be 0-100")
        self._score = value

s = Student(85) //Python calls the constructor
print(s.score) //85
s.score = 120
```

- Access control refers to controlling or managing how attributes and methods are accessed form outside a class.
- Encapsulation is the practice of combining data and the methods that operate on that data within a class, while providing controlled ways to access or modify the data.
- Looks like an attribute, runs getter/setter code
- The setter validates; invalid data is rejected
- _score holds the real value

**Output**

```
85
ValueError: Score must be 0-100
```

# 4: typing.final

```
from typing import final //import final

class Base: //defines a class named Base
    @final //marks the run() method as one that
subclasses should not override
    def run(self):
        print("base")

class Child(Base): //is a child class of Base
    def run(self):
        print("child")

Child().run()
```

- @property let you access a method like an attribute
- @final marks a method that should not be overridden in a child class
- typing.final is the tool used to import the final decorator. Commonly used for type-checking
- @final tells type checkers (mypy, pyright): do not override
- Python does NOT enforce it at runtime
- Same idea applies to @final on a class

**Output**

```
child
```

# 4: Key Takeaways

- ABC + @abstractmethod: required methods, no direct instances
- Encapsulation: data and the methods that guard it live together
- @property: validated, attribute-style access
- @final is a signal for type checkers, not a runtime rule

TOPIC 5

# Python Operators — Part 1

Arithmetic · comparison · logical · bitwise

# 5: Arithmetic Operators

```
print(7 + 2, 7 - 2, 7 * 2)
print(7 / 2)
print(7 // 2)
print(7 % 2)
print(7 ** 2)
print(-7 // 2)
```

- Arithmetic Operators. Ex: +,-,*,/
- Operator acts on operands: 7 + 2
- / true division (float); // floor division
- % remainder; ** power
- Mistake: expecting -7 // 2 to be -3

**Output**

```
9 5 14
3.5
3
1
49
-4
```

# 5: Comparison Operators

```
score = 75
print(score == 75)
print(score != 75)
print(score > 60, score < 60)
print(score >= 75, score <= 70)
```

- Comparisons Operators. Ex: ==, !=, >,>=,<,<=
- Comparisons return True or False
- == compares; = assigns
- Mistake: if score = 75

**Output**

```
True
False
True False
True False
```

# 5: Truth Table (and, or, not)

| A | B | A and B | A or B | not A |
|---|---|---|---|---|
| True | True | True | True | False |
| True | False | False | True | False |
| False | True | False | True | True |
| False | False | False | False | True |

# 5: Short-Circuit Evaluation

```
def check(): //Create a function called check
    print("check called") //print string type
    return True //the string is True
print(False and check())
print(True or check())
print(True and check())
```

- Short-Circuit Evaluation. Ex: && (and), || (or)
- stops if the left side is False
- or stops if the left side is True
- Right side is skipped, so check() is not called

**Output**

```
False
True
check called
True
```

# 5: Bitwise Operators

```
# 5 = 0101, 3 = 0011
print(5 & 3)
print(5 | 3)
print(5 ^ 3)
print(~5)
print(5 << 1, 5 >> 1)

//0101= (0*2^3)+(1*2^2)+(0*2^1)+(1*2^0)=5
//0011= (0*2^3)+(0*2^2)+(1*2^1)+(1*2^0)=3
```

- Bitwise Operators perform operations directly on the individual binary digits of integer values
- & AND: 0001 = 1
- | OR: 0111 = 7
- ^ XOR: 0110 = 6
- ~5 = -6; << doubles, >> halves

**Output**

```
1
7
6
-6
10 2
```

# 5: Key Takeaways

- Arithmetic: / is float division, // floors
- Comparisons produce bool; == is not =
- and / or short-circuit
- Bitwise operators work on binary digits

TOPIC 6

# Operators — Part 2

Assignment · identity · membership · precedence

# 6: Assignment Operators

```
x = 10
x += 5 //x=10+5=15
x -= 3 //x=15-3=12
x *= 2 //x=12*2=24
x //= 5 //x=24/5=4
x **= 2 //x=4^2=16
print(x)
y = 17
y %= 5 //y=17%5=2
y /= 2 //y=2/2=1
print(y)
```

- Assignment Operators are symbols used in programming to assign values to variables. Ex: =, +=, -=,*=,/=,%=,**=,//=,….
- x: 10, 15, 12, 24, 4, 16
- y: 17, 2, 1.0
- /= always gives a float

**Output**

```
16
1.0
```

# 6: == vs is, in, and Conditional

```
a = [1, 2] //create a list
b = [1, 2] //create b list
c = a //assign variable c as a
print(a == b, a is b, a is c)
names = ["Visal", "Dara"]
info = {"name": "Visal"}
print("Dara" in names, "name" in info)
print("x" not in "python")
score = 70
print("Pass" if score >= 60 else "Fail")

//”a==b” is true because a is comparing its value with b
//”a is b” is false because a’s identity and b’s identity
are different even though their values are the same.
//”a is c” is true because we already assigned c as a
which mean c’s value and identity is a
```

- Key words “in” and “not in” are called Membership. “is” and “is not” are called Identity. “x if condition else y” is called Conditional Expressions
- == compares values; is compares identity
- in on a dict checks keys
- Use is None to test for None
- Conditional: A if cond else B

**Output**

```
True False True
True True
True
Pass
```

# 6: Precedence and Associativity

```
print(2 + 3 * 4) //4*3 and then plus 2
print((2 + 3) * 4) //2+3 then multiply by 4
print(20 - 5 - 3) //evaluate right to left
print(2 ** 3 ** 2) //3^2=9 then 2^9
print(not 1 > 2 and 3 < 4) //Obviously
```

- Operator Precedence dictates which operator is evaluated first.
- Operator Associativity dictates the direction of evaluation.
- Order: ** then * / // % then + - then comparisons then not, and, or
- Subtraction: left to right
- ** : right to left
- Parentheses improve readability

**Output**

```
14
20
12
512
True
```

# 6: Key Takeaways

- Assignment operators update a variable in place
- == is equal value; is is same object
- in / not in test membership (dict keys by default)
- Use parentheses when in doubt

TOPIC 7

# Decision-Making Statements — Part 1

if · if-else · if-elif-else

# 7: Grading with if-elif-else

```
score = 85 //set score to 85

if score >= 90:
    grade = "A"
elif score >= 80: //condition matches
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
else:
    grade = "F"

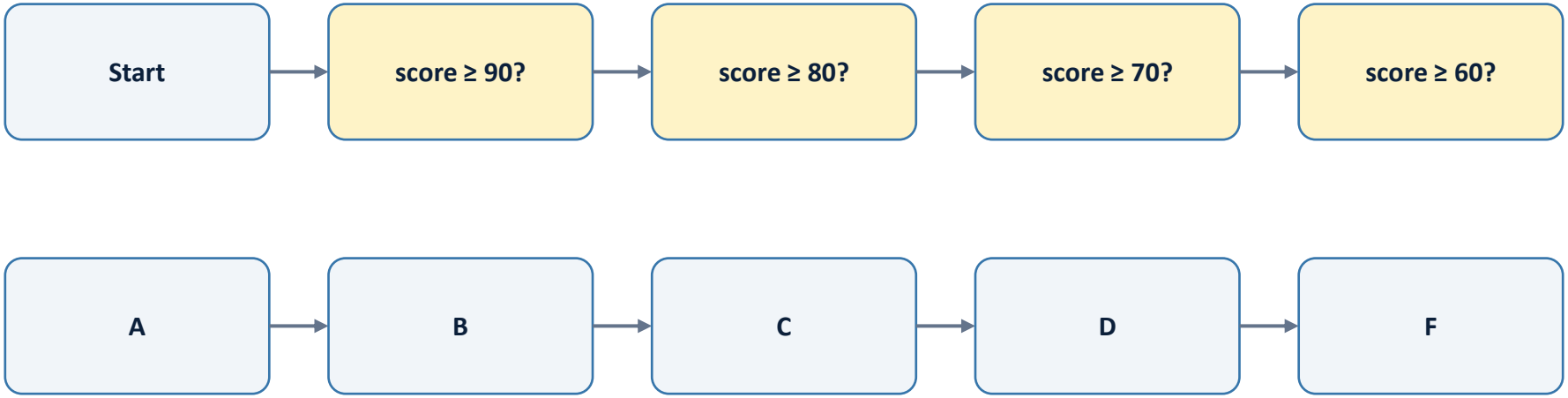
print("Grade:", grade)
```

- If condition is used when we want to set up the system that match with the condition we want. Elif is used when we want another result beside the if condition. Else is used when the previous condition didn't match.
- Conditions are tested top to bottom; first True wins
- Colon and indentation define each block
- 85 fails >= 90, passes >= 80

**Output**

```
Grade: B
(95 → A, 72 → C, 40 → F)
```

# 7: Grading Flowchart




Each 'no' moves right; the first 'yes' gives the grade; none gives F.

# 7: Order Matters (Common Mistake)

```
score = 95
if score >= 60:
    grade = "D"
elif score >= 90:
    grade = "A"
print(grade)

//it leads to the first condition, even
though both conditions are true. Because it
already matched with the first condition
and it skip other condition to print(grade)
```

- Most specific condition must come first
- 95 matches >= 60 first
- Fix: test >= 90 first

**Output**

D

# 7: Combining Conditions and Nesting

```
registered = True //consider registered is True
attendance = 80 //assign attendance to 80
if registered and attendance >= 75:
    print("Eligible")
if registered:
    if attendance < 75:
        print("Low attendance")

//it matches with the first if, because python
know “registered” is true and attendance>=75
which also true, both combine with the and
operator which true&&true=true. When it matches
the condition, it will skip other condition and
will show the output print
```

- and, or, not combine conditions
- Nested if: inner test runs only if the outer is True
- Prefer a combined condition when simpler

**Output**

```
Eligible
```

# 7: Key Takeaways

- if / elif / else choose one branch
- Indentation is the syntax
- Order of conditions matters
- Mistake: = instead of ==

TOPIC 8

# Decision-Making Statements — Part 2

match-case · conditional expressions · menus

# 8: match-case

```
def menu(choice): //function menu with argument choice
    match choice:
        case 1:
            return "Add student"
        case 2:
            return "View students"
        case 3:
            return "Exit"
        case _: //if choice besides 1,2 or 3 it will
choose this option
            return "Invalid option"
print(menu(2))
print(menu(9))
```

- match compares a value against case patterns
- case _ is the wildcard (everything else)
- Needs Python 3.10+
- Use match for many fixed choices; if-elif for ranges

**Output**

```
View students
Invalid option
```

# 8: Conditional Expression and Choosing

- label = "Pass" if score >= 60 else "Fail"
- Good for one simple value
- Avoid chains: use if-else for readability
- match-case: clear for fixed choices
- if-elif-else: best for ranges like grades

# 8: Menu-Driven Program

```
students = []
while True:
    print()
    print("Student Management System")
    print("1. Add student")
    print("2. View students")
    print("3. Exit")
    choice = input("Choose an option: ")
    if choice == "1":
        name = input("Enter student name: ")
        students.append(name)
        print("Student added successfully.")
    elif choice == "2":
        if students:
            for student in students:
                print(student)
        else:
            print("No students found.")
    elif choice == "3":
        print("Goodbye!")
        break
    else:
        print("Invalid option.")
```

- while True repeats the menu
- input() returns text, so compare with "1"
- break leaves the loop
- for + if + list work together

**Output**

(see next slide)

# 8: Sample Interaction

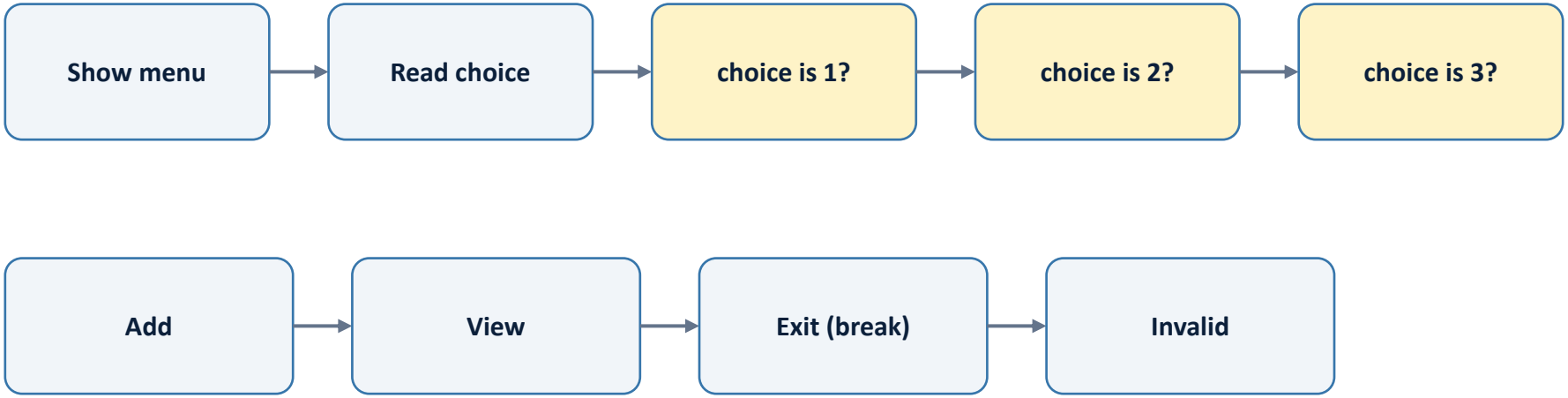
```
Choose an option: 2
No students found.
Choose an option: 1
Enter student name: Dara
Student added successfully.
Choose an option: 2
Dara
Choose an option: 9
Invalid option.
Choose an option: 3
Goodbye!
```

- Empty list is falsy, so 'No students found.' appears first
- Invalid input is handled by else

**Output**

(user types after each prompt)

# 8: Menu Flowchart




The loop returns to the menu after every action except Exit.

# 8: Key Takeaways

- match-case suits fixed choices; case _ handles the rest
- Conditional expressions suit one-line choices
- Menus combine loops, conditions, lists and input

TOPIC 9

# Python Loops

while · for · range · break · continue · pass

# 9: while and range()

```
count = 1
while count <= 5: //1<5 so it loop till count=5
    print(count)
    count += 1 //plus 1 every time count<=5
for i in range(3): //in range starts with 0 value
    print("i =", i) //show 3 rows that start with 0
for n in range(2, 10, 3): //start stop step
    print(n)
```

- while repeats while the condition is True
- Forgetting count += 1 makes an infinite loop
- range(3) gives 0, 1, 2
- range(2, 10, 3) gives 2, 5, 8

**Output**

```
1 2 3 4 5 (one per line)
i = 0
i = 1
i = 2
2
5
8
```

# 9: Iterating Collections

```
names = ["Visal", "Dara"] //list
for name in names: //print all name in list
    print(name)
for ch in "Hi": //print every letter in “Hi”
    print(ch)
for n in (1, 2): //print every number in tuple
    print(n)
info = {"name": "Visal", "age": 20} //dictionary
for key, value in info.items():
    print(key, value)
//printing key and value from info
```

- Lists, tuples and strings give items
- A dict gives keys; .items() gives pairs

**Output**

```
Visal
Dara
H
i
1
2
name Visal
age 20
```

# 9: Total and Average

```
scores = [85, 72, 90, 68, 95] //list

total = 0 //set total to 0

for score in scores: //calculate total in list
    total += score //total=0+(value in scores)

average = total / len(scores) //formula
                //len=length in scores is 5
print("Total:", total)
print("Average:", average)
```

- total starts at 0
- Each pass adds one score
- len() counts items

**Output**

```
Total: 410
Average: 82.0
```

# 9: break, continue, pass

```
names = ["Visal", "Dara", "Sophea"]
for name in names:
    if name == "Dara":
        print("Found Dara")
        break
for n in range(1, 6):
    if n == 3:
        continue //skip 3
    print(n)
def todo():
    pass
```

- break: stop the loop now
- continue: skip to next item
- pass: placeholder, does nothing

**Output**

```
Found Dara
1
2
4
5
```

# 9: Nested Loops

```
for i in range(1, 4):
    for j in range(1, 4):
        print(i * j, end=" ")
    print()
```

- Inner loop runs fully for each outer pass
- end=" " keeps output on one line

**Output**

```
1 2 3
2 4 6
3 6 9
```

# 9: do-while Pattern

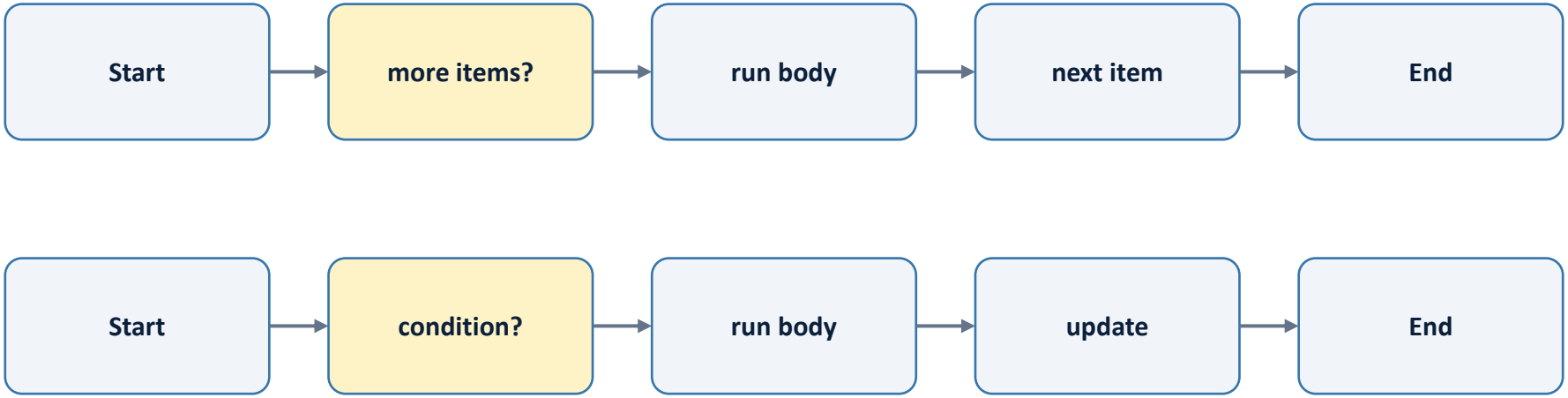
```
while True:
    value = int(input("Enter 1-5: "))
    if 1 <= value <= 5:
        break
    print("Try again")
```

- Python has no native do-while statement
- while True + break runs the body at least once
- Infinite loop prevention: ensure a break or changing condition

**Output**

```
Enter 1-5: 9
Try again
Enter 1-5: 3
```

# 9: for and while Flowcharts




Top row: for loop (items). Bottom row: while loop (condition, then back to the check).

# 9: Key Takeaways

- for: iterate a collection or range
- while: repeat until a condition changes
- break / continue / pass control the flow
- Always make sure a while loop can end

# How the Nine Topics Connect

- Data types hold values; variables and attributes name them
- Classes bundle attributes and methods; conventions guard them
- Operators compute; conditions decide; loops repeat
- Together: records, rules, menus and reports

# Integration 1/2: Student Class

```
class Student:
    def __init__(self, name, scores):
        self.name = name
        self.scores = scores

    def average(self):
        return sum(self.scores) / len(self.scores)

    def grade(self):
        avg = self.average()
        if avg >= 80:
            return "A"
        elif avg >= 60:
            return "B"
        return "C"
```

- Attributes, methods, operators, if-elif

**Output**

```
(class definition)
```

# Integration 2/2: Records and Search

```
students = [
    Student("Visal", [85, 90, 78]),
    Student("Dara", [60, 55, 70]),
]
for s in students:
    print(s.name, round(s.average(), 1), s.grade())
name = input("Search: ")
for s in students:
    if s.name == name:
        print("Found", s.name)
        break
else:
    print("Not found")
```

- for-else: else runs if no break happened
- Extension: validate empty input with if not name.strip()

**Output**

```
Visal 84.3 A
Dara 61.7 B
Search: Dara
Found Dara
```

# Final Summary

- Know your types and conversions
- Use classes, attributes and conventions deliberately
- Operators and precedence: use parentheses
- Decisions and loops turn data into programs

# References (Python Documentation)

- AI Platforms such as (ChatGPT, Claude and Gemini)
  - ChatGPT: understanding the code and how it works.
  - Gemini: for understanding the definition and how it is applied in code.
  - Claude: for creating slide presentation and examples of code
- W3school to explore the core understanding of Python OOP.

# Thank You

Keep coding, keep experimenting.